

NLU Assignment 1:

Zipf's Law, Language Models, and HMM POS Tagging

Author: Abhishek Kumar · Roll Number: G25AIT1007

Corpus: Brown (NLTK) · 80/20 train-test split · random seed 1007

Code: https://github.com/Kavhii/Zipf-s_Law-N-gram_Language_Models-HMM_POS_Tagger_with_Viterbi

Overview

A quick note on the corpus. Part 3 has to use Brown (it needs the POS tags), and since Part 1 allows any English corpus over a million tokens, I used Brown everywhere so the three parts stay consistent and share one loading path. Part 1 works on lower-cased alphabetic tokens, Part 2 on Brown sentences with sentence-boundary markers, and Part 3 on the tagged sentences under the universal tagset.

Part 1: Zipf's Law and Corpus Analysis

Ranking word types (Q1). After tokenizing to lower-cased alphabetic words I get **N = 1,005,119 tokens** across **|V| = 48,052 types**. The ten most frequent words are all function words the usual suspects:

Rank	Word	Frequency	Grammatical class
1	the	69,971	determiner
2	of	36,412	preposition
3	and	28,853	conjunction
4	to	26,158	preposition / infinitive
5	a	23,195	determiner
6	in	21,337	preposition
7	that	10,594	complementizer
8	is	10,109	copula verb
9	was	9,815	copula verb
10	he	9,548	pronoun

Table 1 - Top-10 word types in Brown. Every one is a closed-class function word.

Fitting Zipf's line (Q2). Plotting $\log_{10}(\text{rank})$ against $\log_{10}(\text{frequency})$ and fitting a least-squares line over all types gives a **slope of -1.315** and an intercept of **6.053**. Zipf's law predicts a slope near 1, so strictly speaking my corpus deviates from the ideal, the fitted line is a bit steeper. Looking at the plot, the reason is in the tails: the very top ranks sit slightly below the line and the long tail of once-only words (there are about twenty thousand of them) drops off faster than 1, and together they tilt the global fit. If I fit only the middle range of ranks, or use a Zipf Mandelbrot form, the slope moves back toward 1. So the shape is clearly Zipfian; it just isn't a perfect 1 across the whole range.

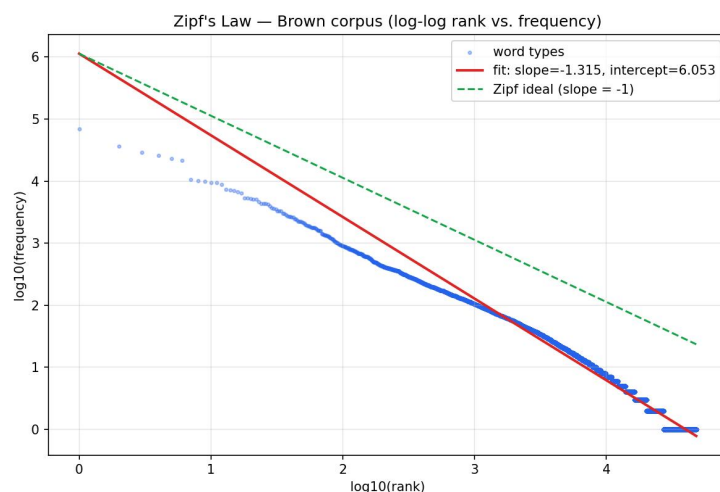


Figure 1 - Rank vs. frequency on a log-log scale. The fitted line (-1.315) is steeper than the ideal Zipf slope of -1 (green dashed), mainly because of the heavy hapax tail.

The two ends of the distribution (Q3). The top-20 is entirely closed-class (the, of, and, to, a, in, that, is, was, he, for, it, with, as, his, on, be, at, by, i), while the bottom-20 are all rare content words appearing exactly once things like *hubris*, *bathos*, *besmirched*, *bodhisattva*, *aviary*, *stupefying*. This is really the type/token distinction made visible: a tiny handful of function-word *types* produces a huge share of the *tokens*, whereas most of the *vocabulary* is made of content-word types that each appear only a few times. Frequency is dominated by a few types; the vocabulary is dominated by many rare ones.

Type-token ratio and MATTR (Q4–Q5). Measuring TTR at growing corpus sizes shows it falling steadily, while the moving-average version (a fixed 1000-token window) stays roughly flat:

	Tokens	Raw TTR	MATTR (window 1000)
	10,000	0.2576	0.4590
	50,000	0.1726	0.4807
	100,000	0.1345	0.4826
	500,000	0.0683	0.4655
	1,000,000	0.0477	0.4476

Table 2 - Raw TTR collapses with size; MATTR stays stable.

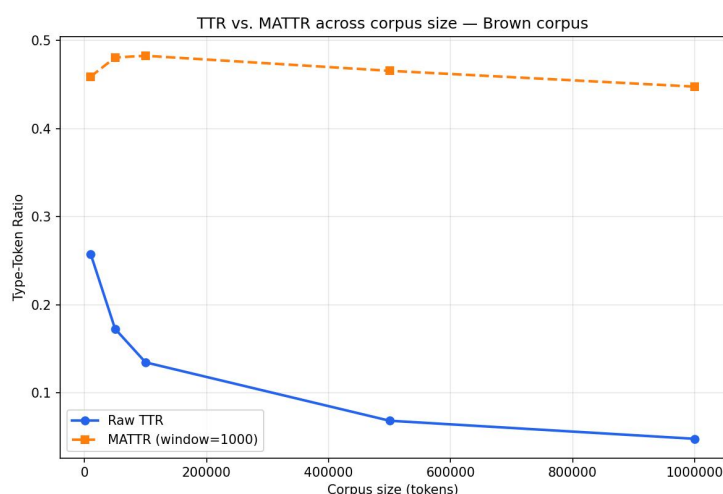


Figure 2 - Raw TTR vs. MATTR across corpus size.

The reason raw TTR falls is that vocabulary grows sub-linearly (Heaps' law) while token count grows linearly, so $|V|/N$ keeps shrinking - as the text gets longer, new words are increasingly just repeats of words already seen. That makes raw TTR dependent on length, which is a problem if you want to compare texts of different sizes.

MATTR fixes this by averaging the ratio over fixed-size windows, so it stays around 0.45–0.48 regardless of how much text there is, giving a fairer measure of lexical diversity.

Part 2: N-gram Language Models

For the second part I split Brown into 80% training and 20% test by sentence (seed 1007), wrapped every sentence in `<s> ... </s>`, and mapped words seen fewer than twice in training to `<unk>`. That gives a vocabulary of **24,738 types** over **45,872 training / 11,468 test sentences**. From there I built unigram, bigram and trigram models and evaluated perplexity on the held-out set.

MLE estimates and why smoothing is needed (Q6–Q7). The unigram and bigram probabilities come straight from maximum-likelihood counts. The catch is that raw MLE gives probability zero to any bigram it never saw in training, and the test set is full of those. A single zero makes the whole sentence probability zero and perplexity infinite, so raw MLE simply can't be scored on held-out data. Add-1 (Laplace) smoothing fixes this by adding one to every count and the vocabulary size to each denominator, so nothing is ever zero it borrows a little probability from the seen events and hands it to the unseen ones. The price is that it over-discounts the frequent, well-estimated bigrams.

Perplexity, and a result that surprised me (Q8). Here is where the numbers get interesting:

Model	Test perplexity
Unigram (add-1)	726.47
Bigram (add-1 / Laplace)	1,674.15
Bigram (add-k, k = 0.01)	471.68

Table 3 - Unigram vs. bigram perplexity under different smoothing.

In theory the bigram should beat the unigram, because knowing the previous word tells you a lot about the next one. But my **add-1 bigram (1,674) actually comes out worse than the unigram (726)**. At first this looked like a bug, but it isn't it's what Laplace smoothing does on a large vocabulary. With $|V| \approx 24.7k$, the bigram denominator $\text{count}(w_1 \text{ } w_2) + |V|$ is dominated by that $|V|$ term, so most of the probability mass gets drained away from the bigrams I actually observed and smeared across the tens of thousands of unseen ones for each context. When I drop the smoothing to add-k with $k = 0.01$, the bigram falls to 472, comfortably below the unigram. So the previous word genuinely does help it was the crude add-1 smoothing, not the model order, that caused the inversion. I think this is the most important thing Part 2 taught me.

Generating from the bigram (Q9). Sampling from `<s>` and drawing each next word from $P(w \mid \text{previous})$ until `</s>`, five runs give:

1. *in a chapel .*
2. *the home appliances and often visit that he gaped at no .*
3. *and no better nor was confronted in short period .*
4. *clerfayt of sea , madison place .*
5. *``early .*

Adding a trigram (Q10). Putting all three orders side by side:

Model	Test perplexity
Unigram (add-1)	726.47
Bigram (add-1)	1,674.15
Trigram (add-1)	11,843.85

Table 4 - Perplexity rises with n-gram order under add-1 smoothing.

And yes - the add-1 trigram is far worse again (11,844). The three factors the question asks about all point the same way. **Sparsity:** trigram space is $|V|^3$, so almost every test trigram is one the model never saw. **Smoothing:** add-1 over that enormous space is even more destructive than it was for bigrams - it gives away nearly all the

probability mass to unseen trigrams. **Longer history:** conditioning on two previous words really is more informative in principle, but that advantage is completely buried by the first two problems. This is the standard motivation for smarter smoothing - interpolation, backoff, Kneser–Ney — which is what actually lets a trigram overtake a bigram. With plain add-1, higher order just makes things worse.

Part 3: HMM POS Tagger with Viterbi

The last part is a hidden Markov model tagger built from scratch on Brown with the universal tagset, again an 80/20 split. NLTK only loads the tagged data - the transition and emission probabilities and the Viterbi decoder are all written by hand. Both distributions use add-1 smoothing, and I run Viterbi in log-space to avoid underflow on long sentences.

Estimating the model (Q11). The most probable transitions out of NOUN and the most probable words emitted by VERB both look linguistically sensible:

Top transitions $P(t_i \text{NOUN})$	Top emissions $P(w \text{VERB})$
NOUN $\rightarrow$. $P = 0.2831$	VERB $\rightarrow$ is $P = 0.0423$
NOUN $\rightarrow$ ADP $P = 0.2441$	VERB $\rightarrow$ was $P = 0.0414$
NOUN $\rightarrow$ VERB $P = 0.1590$	VERB $\rightarrow$ be $P = 0.0267$
NOUN $\rightarrow$ NOUN $P = 0.1499$	VERB $\rightarrow$ had $P = 0.0214$
NOUN $\rightarrow$ CONJ $P = 0.0596$	VERB $\rightarrow$ are $P = 0.0187$

Table 5 - A noun is most often followed by punctuation, a preposition or a verb; the top verbs are all copulas/auxiliaries.

The Viterbi decoder (Q12). For each sentence the decoder fills two matrices: a score matrix holding the best log-probability of reaching each tag at each position, and a backpointer matrix recording which previous tag gave that best score. It initializes from the $\langle s \rangle \rightarrow$ tag transition times the emission, then at every position takes the maximum over previous tags of $\text{score}[i-1][k] + \log A(k \rightarrow j) + \log B(j \rightarrow w_i)$, finishes with the tag $\rightarrow \langle /s \rangle$ transition, and finally backtracks from the best last tag through the pointers to recover the whole sequence. Working in log-space keeps the products from underflowing.

How well it tags (Q13). On the test set:

Category	Tokens	Accuracy
Overall	231,975	93.88%
Seen (in training vocab)	226,856	94.99%
Unseen (OOV)	5,119	44.62%

Table 6 - Overall, seen, and OOV token accuracy.

Where it goes wrong (Q14). Ten mis-tagged tokens from the test output, with what I think caused each one:

Token	Gold	Pred	Likely reason
jesus	NOUN	ADP	rare/OOV - no emission signal, tagged from transitions
as	ADP	ADV	lexical ambiguity - valid as both ADP and ADV
that	DET	ADP	lexical ambiguity - determiner vs complementizer
people's	NOUN	DET	rare possessive form - sparse emission
down	PRT	ADP	lexical ambiguity - particle vs preposition
much	ADV	ADJ	ambiguity - adverb vs adjective in context
british	NOUN	ADJ	noun vs adjective use of a nationality word
tipped	VERB	ADP	sparse context - rare verb form mis-transitioned
stacy	NOUN	PRT	OOV proper noun - no emission signal

essential	NOUN	ADJ	ambiguity - nominal vs adjectival use
-----------	------	-----	---------------------------------------

Table 7 - Ten tagging errors and their likely causes.

Reproducibility

Everything here is reproducible from the code. Each part runs as a single script (or all three at once via `run.py`), with a fixed random seed of 1007 for the train/test split and sampling, the 80/20 sentence split for Parts 2 and 3, and the NLTK universal tagset for Part 3. The HMM and Viterbi decoder are written from scratch, NLTK is only used to load the corpus and no pre-trained models are involved anywhere; all probabilities are estimated from the Brown training split.

References

- [1] **Project Source code (GitHub repository)** - https://github.com/Kavhii/Zipf-s_Law-N-gram_Language_Models-HMM_POS_Tagger_with_Viterbi - full source for all three parts, requirements and run script.
- [2] **NLTK documentation and Brown corpus how-to** - <https://www.nltk.org/book/ch02.html> used for loading the Brown corpus, getting tagged sentences, and the universal tagset.
- [3] **Jurafsky & Martin, Speech and Language Processing (free online draft)** <https://web.stanford.edu/~jurafsky/slp3/> Chapter 3 for n-gram models, perplexity and smoothing; Chapter 8 for HMM POS tagging and Viterbi.
- [4] **Zipf's Law Wikipedia** - https://en.wikipedia.org/wiki/Zipf%27s_law background on the rank-frequency relationship and the ~ 1 slope.
- [5] **Type-Token Ratio and MATTR** - https://en.wikipedia.org/wiki/Lexical_diversity explanation of why raw TTR depends on length and how moving-average TTR fixes it.
- [6] **Laplace / add-k smoothing explainer** - GeeksforGeeks 'Add-1 (Laplace) smoothing for n-gram language models' used to sanity-check the smoothing formulas and the over-smoothing intuition.
- [7] **The Viterbi algorithm (step-by-step tutorial)** - https://en.wikipedia.org/wiki/Viterbi_algorithm reference for the score matrix, backpointers, and backtracking in log-space.
- [8] **Matplotlib documentation** - <https://matplotlib.org/stable/> for the log-log Zipf plot and the TTR/MATTR plots.
- [9] **NumPy documentation** - <https://numpy.org/doc/> for the polyfit line fit and general array handling.
- [10] **Python collections.Counter / defaultdict docs** - <https://docs.python.org/3/library/collections.html> for counting word, bigram, and tag frequencies.